Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SOFTWARE ENGINEERING LAB (BCSE301P)

SecuExam (Secure Exam Paper Distribution)

DONE BY : AKSHAT SPARSH (23BDS0149)

OTHER TEAM MEMBERS:
TANUJ KAPOOR (23BCT0030)
KARTIK KANDWAL (23BCT0060)

NOTE: ALL DIAGRAMS HAVE BEEN GENERATED FROM PLANTUML.COM, BUT I JUST DID NOT COPY PASTE, THIS NEEDED LOGICAL UNDERSTANDING FROM MY END AS WELL.

BELOW IS THE LINK I USED:
https://www.plantuml.com/plantuml/uml/SoWkIImgAStDuNBAJrBGjLDmpCbCJbMmKiX8pSd9vt98pKi1IW80

OUR PROJECT: SECUEXAM

Project Overview: SecuExam (Secure Exam Paper Distribution)

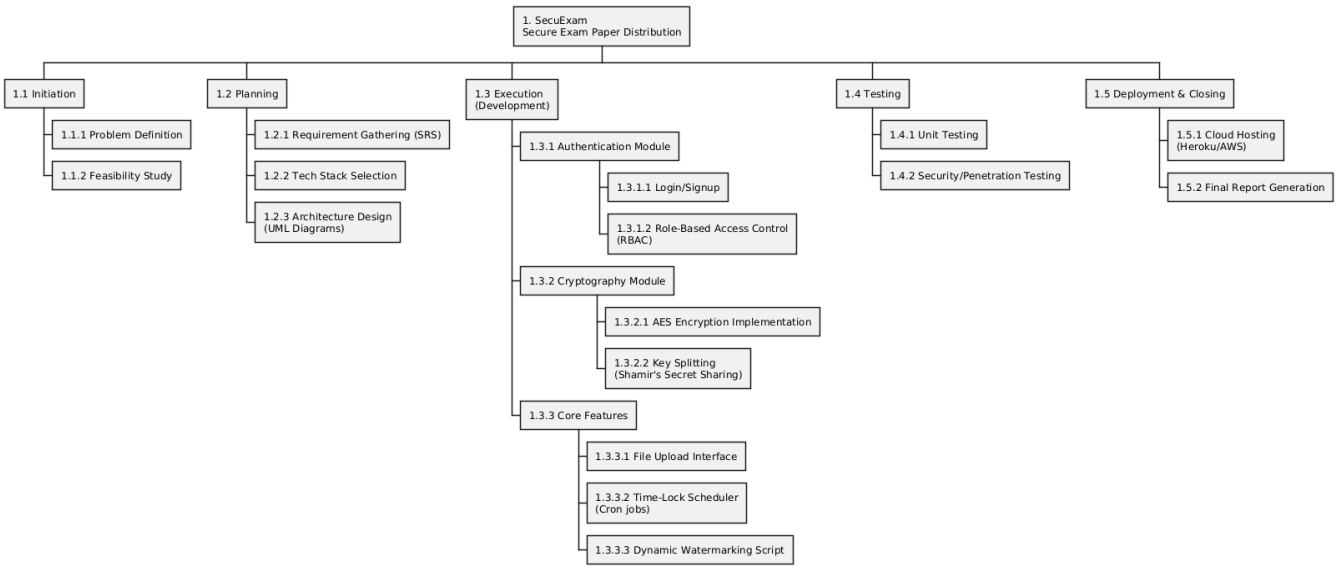
SecuExam is a secure digital platform designed to stop exam paper leaks. Instead of transporting papers physically in vans, which is risky and expensive, our system allows universities to send exam papers to colleges over the internet securely.

It protects the papers using three main features:

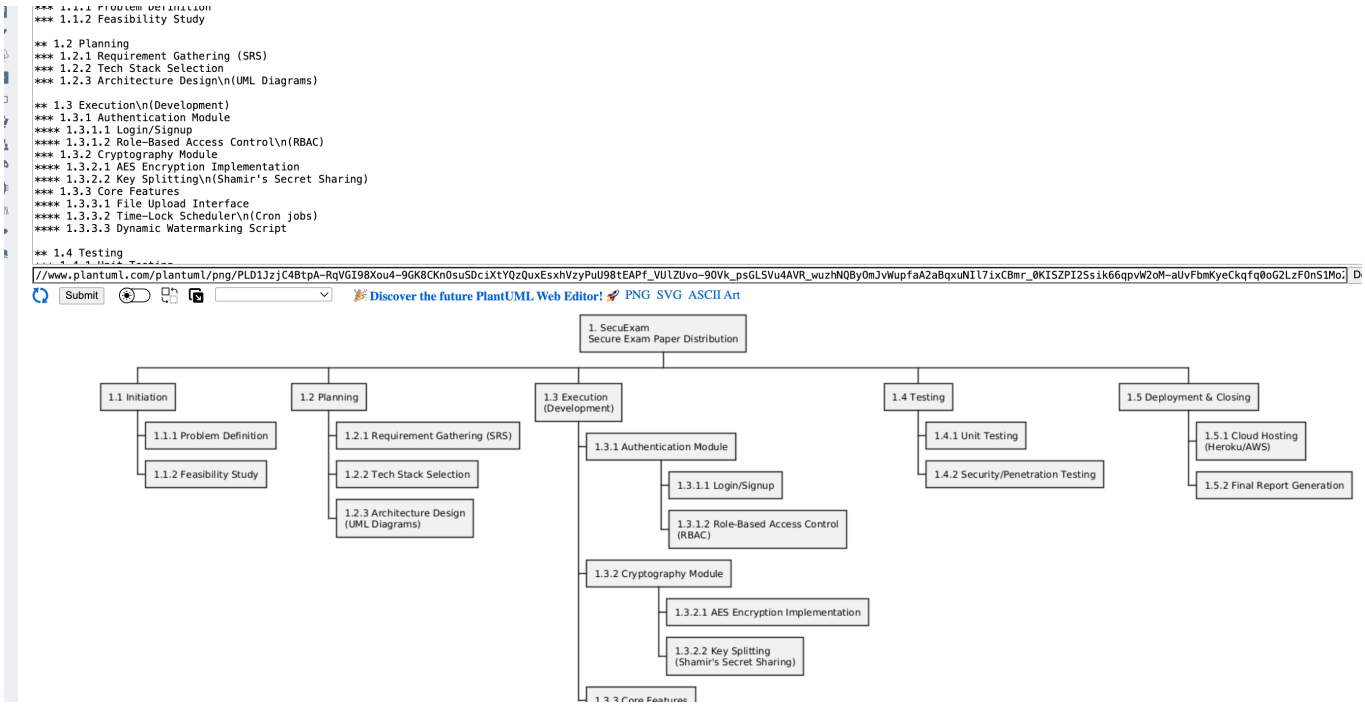
- **Encryption:** The moment a paper is uploaded, it is scrambled into a code that no one can read.
- **Time-Lock:** The system acts like a digital time vault—colleges are technically blocked from downloading the paper until exactly 30 minutes before the exam starts.
- **Watermarking:** If someone downloads the paper, their IP address is stamped on every page, so if it leaks, we know exactly who did it.

In short: We replace risky physical delivery with a hacked-proof digital delivery system.

WBS DIAGRAM OF OUR PROJECT:



RELEVANT SCREENSHOT OF CODE AND OUTPUT OF WBS:

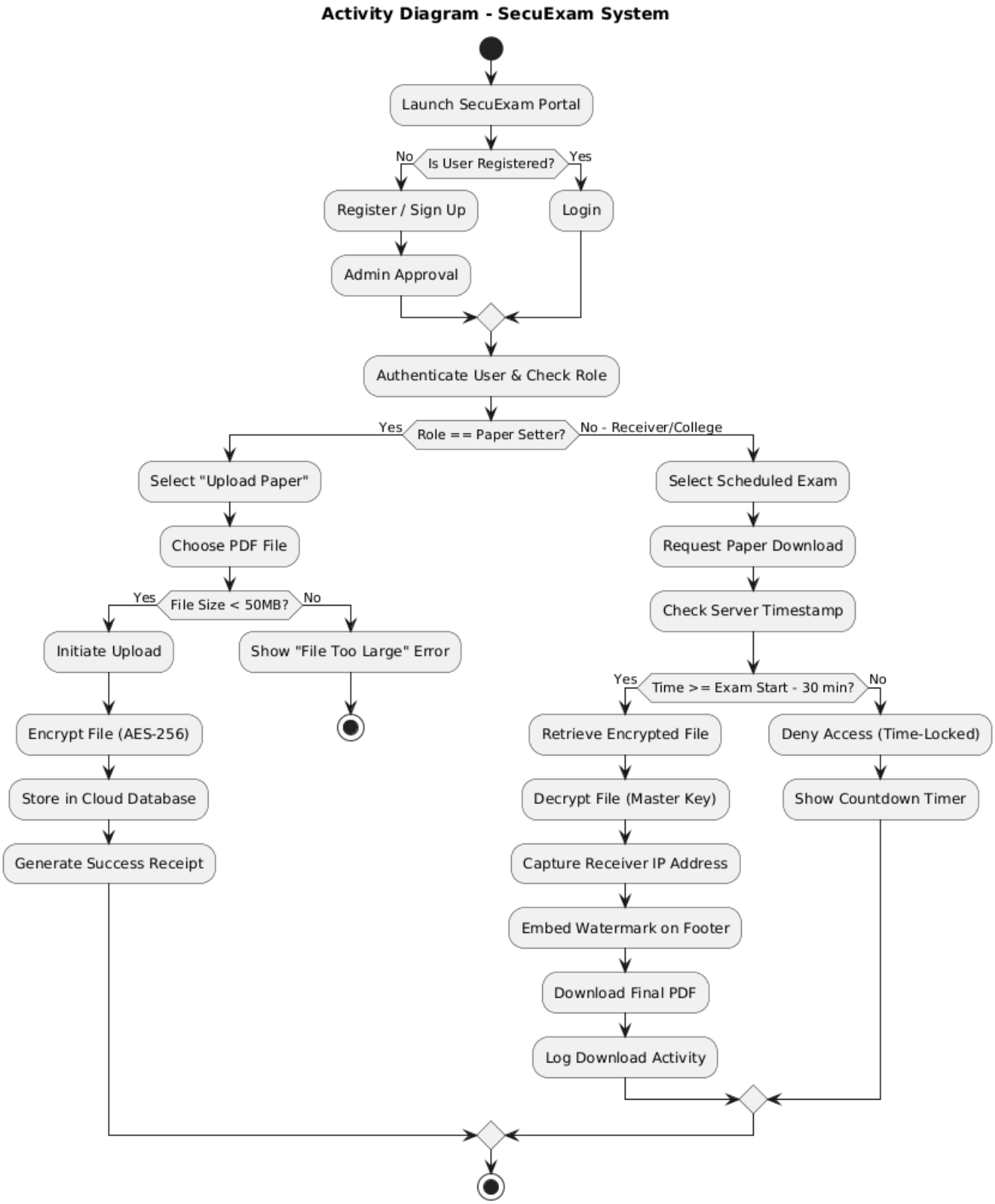


Work Breakdown Structure (WBS)

The WBS organizes the project into five distinct phases to ensure all security and development tasks are tracked.

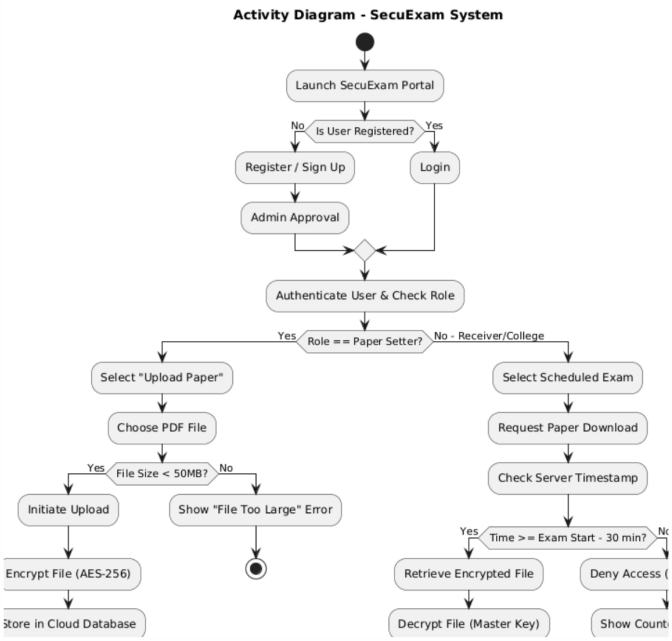
- **Initiation:** Focuses on defining the problem (paper leakage) and conducting feasibility studies to confirm the technical approach.
- **Planning:** Involves gathering requirements (SRS), selecting the tech stack (Python/React), and designing the system architecture.
- **Execution (Development):** The core phase where the **Authentication**, **Cryptography** (AES & Shamir's Secret Sharing), and **Core Features** (Time-Lock & Watermarking) are built.
- **Testing:** Critical for security, this includes Unit Testing and specific Penetration Testing to find vulnerabilities.
- **Deployment:** Covers hosting the system (cloud/AWS) and generating the final project reports.

ACTIVITY DIAGRAM FOR OUR PROJECT:



RELEVANT SCREENSHOT OF CODE AND OUTPUT OF ACTIVITY DIAGRAM:

```
@startuml
title Activity Diagram - SecuExam System

start
:Launch SecuExam Portal;

if (Is User Registered?) then (No)
    :Register / Sign Up;
    :Admin Approval;
else (Yes)
    :Login;
endif

:Authenticate User & Check Role;

if (Role == Paper Setter?) then (Yes)
    :Select "Upload Paper";
    :Choose PDF File;

    if (File Size < 50MB?) then (Yes)
        :Initiate Upload;
        :Encrypt File (AES-256);
        :Store in Cloud Database;
        :Generate Success Receipt;
    else (No)
        :Show "File Too Large" Error;
        stop
    endif

else (No - Receiver/College)
    :Select Scheduled Exam;
    :Request Paper Download;

    ' This is the Time-Lock Feature FR-03
    :Check Server Timestamp;

    if (Time >= Exam Start - 30 min?) then (Yes)
        :Retrieve Encrypted File;
        :Decrypt File (Master Key);

        ' This is the Watermarking Feature FR-04
        :Capture Receiver IP Address;
        :Embed Watermark on Footer;
```

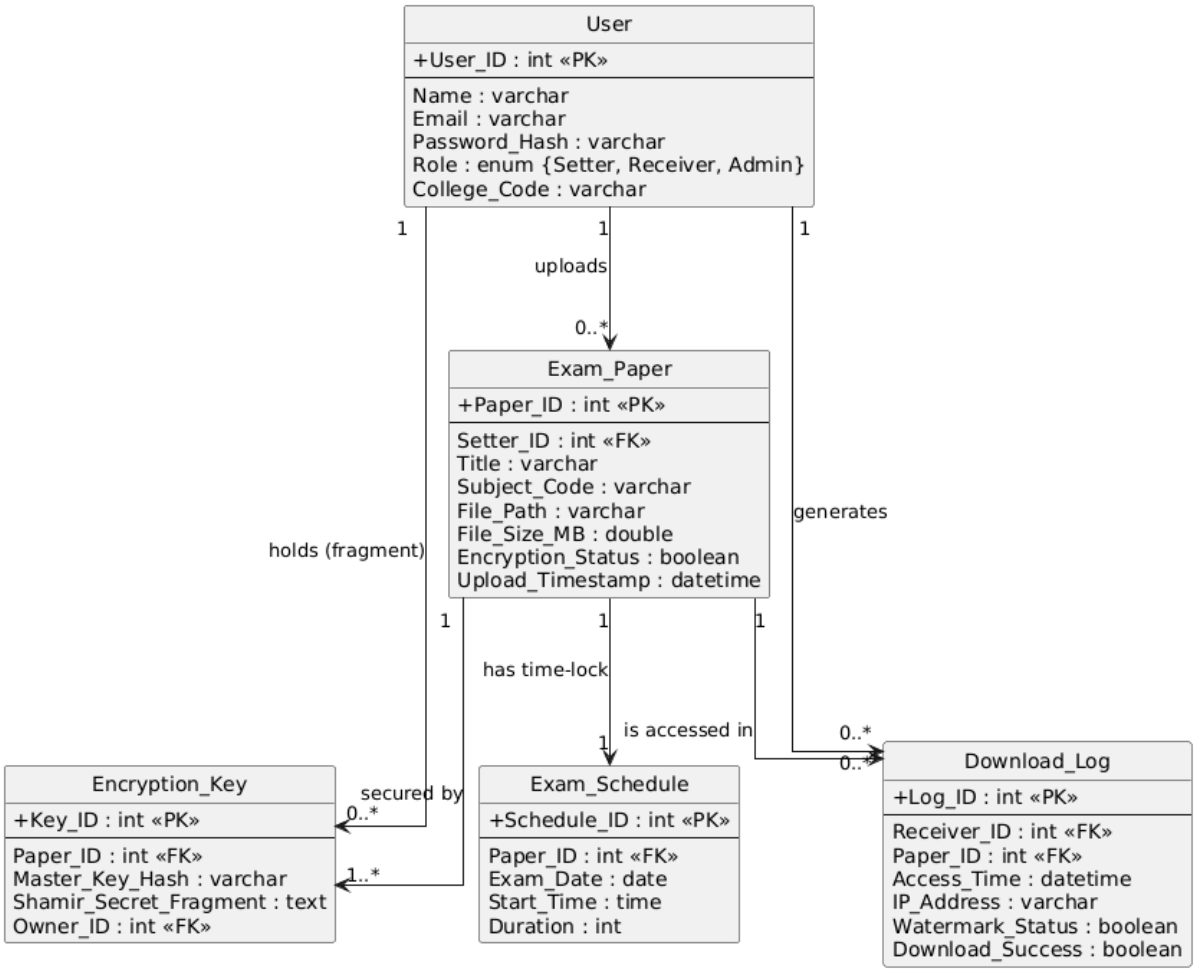


Activity Diagram

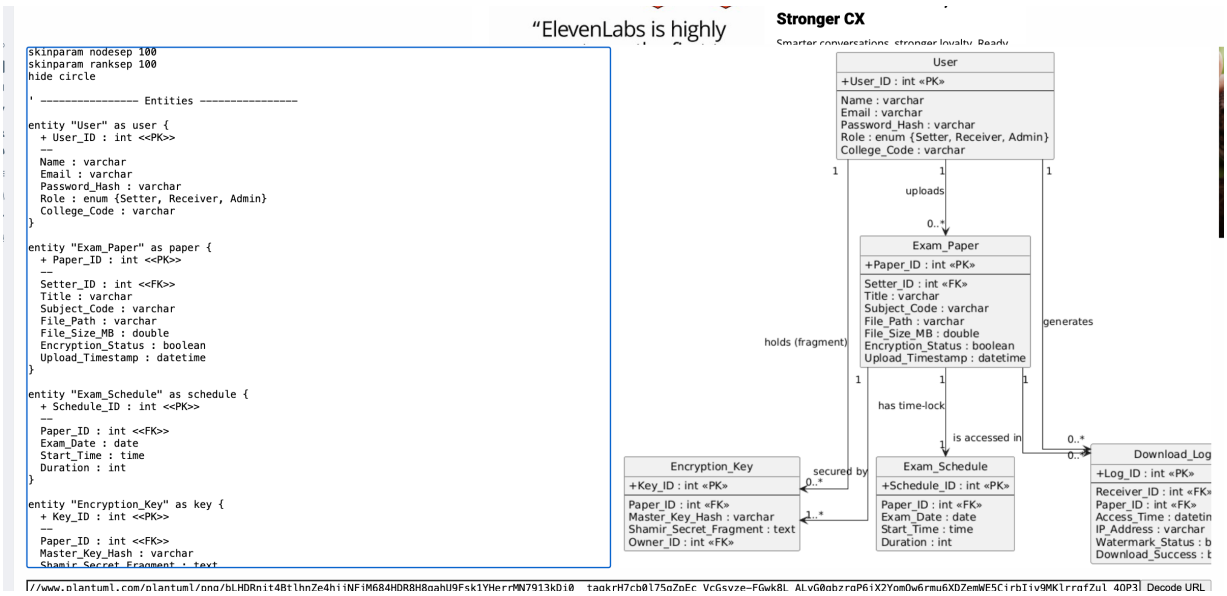
This diagram visualizes the workflow for the two main user roles: the **Paper Setter** and the **Receiver**.

- **Setter Flow:** Starts with authentication, followed by uploading the PDF. The system automatically checks the file size and immediately applies **AES-256 encryption** before storage.
- **Receiver Flow:** Involves logging in and requesting a paper. The system performs a critical **Time-Lock Check** (Server Time vs. Exam Time).
- **Security Steps:** If the time is valid, the system decrypts the file, captures the receiver's IP, embeds it as a **Watermark** on the footer, and finally allows the download.

ER DIAGRAM OF OUR PROJECT:



RELEVANT SCREENSHOT OF CODE AND OUTPUT OF ER:



Entity-Relationship (ER) Diagram

The ER Diagram models the database structure required to support secure paper distribution.

- **User:** Stores details for Setters, Receivers, and Admins, distinguishing roles via a specific field to control access.
- **Exam Paper:** The central entity containing the file path, size (capped at 50MB), and encryption status.
- **Exam Schedule:** Links to the paper to enforce the **Time-Lock**, ensuring downloads only happen 30 minutes before the start time.
- **Encryption Key:** Manages security by storing key fragments (Shamir's Secret Sharing) rather than a single vulnerable key.
- **Download Log:** Tracks every access attempt, recording the receiver's **IP Address** for the watermarking feature and audit trails.

Logical Database Requirements: SecuExam

Project: Secure Exam Paper Distribution System

Module: Backend / Database Design

1. Overview

The database for **SecuExam** is designed to ensure high integrity, confidentiality, and availability of exam materials. It must support role-based access control, store large binary objects (PDF paths), and enforce strict temporal constraints for the "Time-Lock" mechanism[222]. The schema is normalized to **3NF** to reduce redundancy while optimizing for read-heavy operations during exam windows.

2. Logical Schema Architecture

The system requires five core entities to manage users, papers, schedules, security keys, and audit logs.

A. User Management Entity

- **Purpose:** Stores authentication and authorization details for all stakeholders.
- **Fields:**
  - User_ID (PK): Unique identifier (Integer/UUID).
  - Email: Unique contact email used for login (Varchar, Not Null).
  - Password_Hash: Bcrypt-hashed password string; cleartext passwords are strictly forbidden
  - Role: Enumerated type ('Paper Setter', 'Receiver', 'Admin') to enforce RBAC
  - College_Code: Foreign key linking to a master college list (for Receivers).

B. Exam Paper Entity

- **Purpose:** detailed metadata for the uploaded exam papers.
- **Fields:**
  - Paper_ID (PK): Unique identifier.
  - Setter_ID (FK): Links to the User who uploaded the file.
  - File_Path: Secure cloud storage reference (URI) rather than storing BLOBs directly in the DB for performance.
  - File_Size_MB: To enforce the 50MB storage limit constraint.
  - Encryption_Status: Boolean flag (True/False) confirming AES-256 encryption completion.

C. Exam Schedule (Time-Lock) Entity

- **Purpose:** Critical for the "Time-Lock" feature. This table controls *when* a paper becomes accessible.
- **Fields:**
  - Schedule_ID (PK): Unique identifier.
  - Paper_ID (FK): Links to the specific exam paper (One-to-One relationship).
  - Exam_Start_Time: Timestamp (UTC). The application logic uses this to calculate the "Unlock Time" (Start Time - 30 mins)

- Duration: Integer (minutes).

D. Security & Key Management Entity

- **Purpose:** Stores cryptographic fragments to support **Shamir's Secret Sharing**.
- **Fields:**
  - Key_ID (PK): Unique identifier.
  - Paper_ID (FK): Links to the encrypted paper.
  - Key_Fragment: Text/Hex string containing a partial key share.
  - Owner_ID (FK): The specific Admin or trusted user assigned to this fragment

E. Audit & Download Log Entity

- **Purpose:** Tracks every interaction for security audits and watermarking data.
- **Fields:**
  - Log_ID (PK): Unique identifier.
  - Receiver_ID (FK): The user attempting the download.
  - Access_Timestamp: Exact time of the request (used to verify Time-Lock compliance).
  - IP_Address: The IP captured for the **Dynamic Watermark** feature[9].
  - Status: Enumerated ('Success', 'Time-Lock Blocked', 'Failed').

3. Data Integrity & Constraints

1. **Referential Integrity:**
   - A Download_Log cannot exist without a valid User and Exam_Paper.
   - Deleting an Exam_Paper must cascade to delete its associated Exam_Schedule and Encryption_Keys to prevent orphaned security data.
2. **Domain Constraints:**
   - File_Size_MB must be >= 50.0
   - Role must strictly match defined system roles; no undefined users allowed.
3. **Uniqueness:**
   - Each Paper_ID can have only **one** active Exam_Schedule (One-to-One).
   - Email addresses must be unique across the system.

4. Performance & Security Requirements

- **Indexing:** An index must be created on Exam_Schedule.Start_Time and Exam_Schedule.Paper_ID to ensure the Time-Lock check query returns in < 2 seconds, even with 500 concurrent users.
- **Encryption:** Sensitive columns (like Key_Fragment) should be encrypted at rest within the database itself, separate from the file encryption.

REFERENCES OF OUR PROJECT:

References for our project:

- Python "Cryptography" Library Documentation: https://cryptography.io/en/latest/
- OWASP - Guide to Role-Based Access Control (RBAC): https://cheatsheetseries.owasp.org/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.html
- Pressman, R. S. (2010). *Software Engineering: A Practitioner's Approach* (7th ed.). McGraw-Hill. This covers SRS, WBS, project planning, and ITTO processes central to our project.
- Sommerville, I. (2015). Software Engineering (10th ed.). Pearson. It details requirements specification, agile/waterfall lifecycles, and risk management.

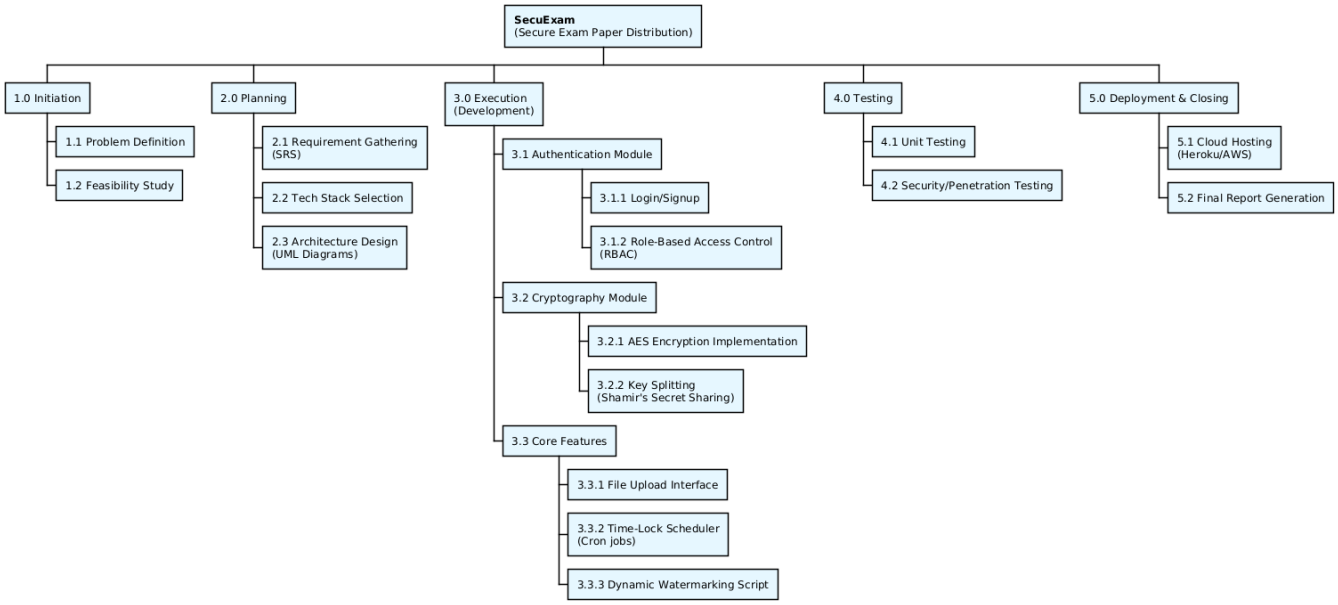
EXISTING PROJECT:

GITHUB LINK: https://github.com/Divijkatyal0406/CrowdQuest

Overall Project Description: CrowdQuest

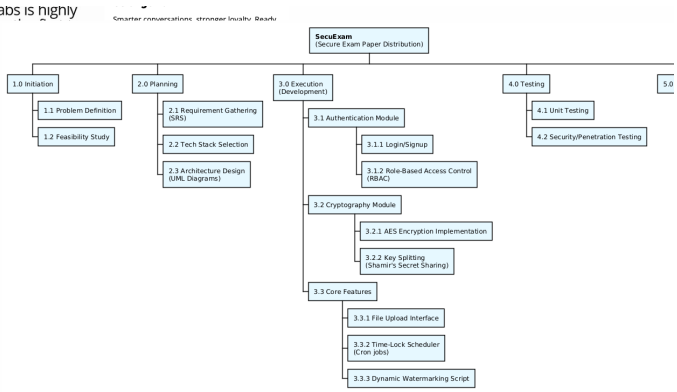
CrowdQuest is a **blockchain-based crowdsourcing platform** designed to create a large, verified question bank and securely distribute exam papers. It allows various stakeholders to contribute questions, which are then vetted by experts. Once a paper is automatically generated from this pool, it is **encrypted and stored on a decentralized network (IPFS/Polygon)** to ensure tamper-proof security and "just-in-time" delivery to exam centers, effectively preventing paper leakage.

WBS DIAGRAM FOR THE REPO PROJECT:



RELEVANT SCREENSHOT OF CODE AND OUTPUT OF WBS:

```
@startwbs
<style>
wbsDiagram {
  BackgroundColor white
  LineColor black
  FontName SansSerif
  node {
    BackgroundColor #E6F7FF
    BorderColor black
  }
}
</style>

* <b>SecuExam</b>\n(Secure Exam Paper Distribution)

** 1.0 Initiation
*** 1.1 Problem Definition
*** 1.2 Feasibility Study

** 2.0 Planning
*** 2.1 Requirement Gathering\n(SRS)
*** 2.2 Tech Stack Selection
*** 2.3 Architecture Design\n(UML Diagrams)

** 3.0 Execution\n(Development)
*** 3.1 Authentication Module
**** 3.1.1 Login/Signup
**** 3.1.2 Role-Based Access Control\n(RBAC)
*** 3.2 Cryptography Module
**** 3.2.1 AES Encryption Implementation
**** 3.2.2 Key Splitting\n(Shamir's Secret Sharing)
*** 3.3 Core Features
**** 3.3.1 File Upload Interface
**** 3.3.2 Time-Lock Scheduler\n(Cron jobs)
**** 3.3.3 Dynamic Watermarking Script

** 4.0 Testing
*** 4.1 Unit Testing
*** 4.2 Security/Penetration Testing

** 5.0 Deployment & Closing
*** 5.1 Cloud Hosting\n(Heroku/AWS)
*** 5.2 Final Report Generation

@endwbs
```

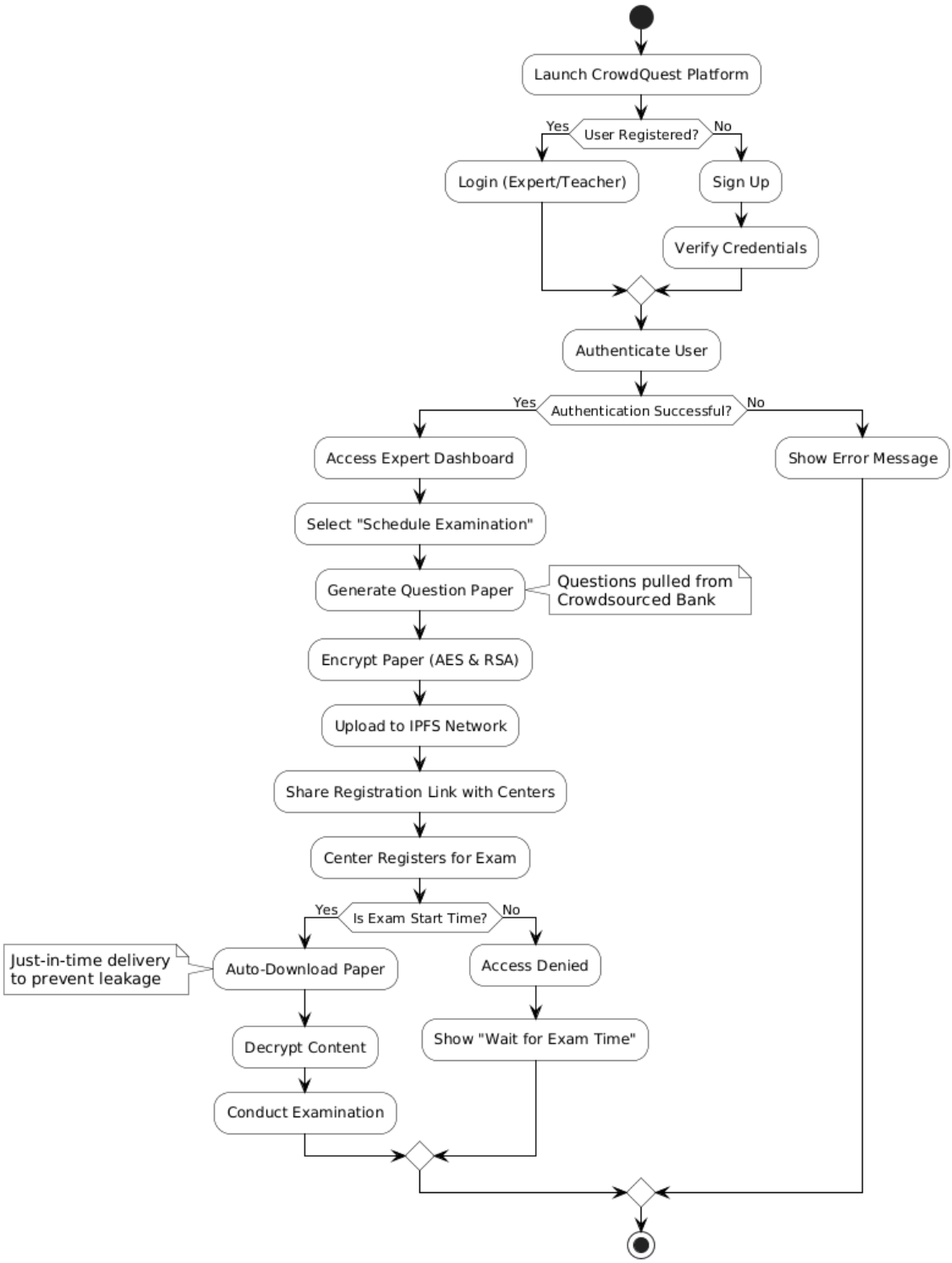


Work Breakdown Structure (WBS) Description

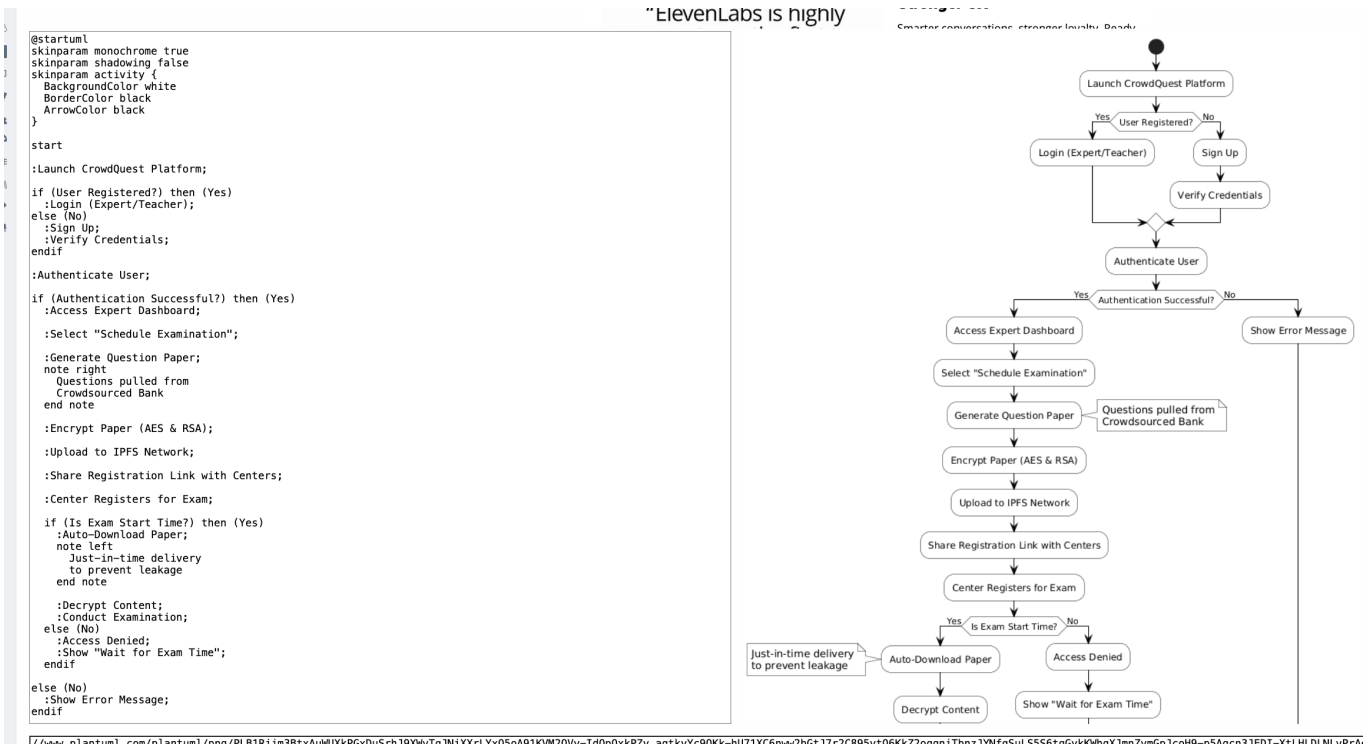
The WBS decomposes the CrowdQuest development into manageable distinct phases, combining Blockchain backend and Web frontend tasks.

- **Phase 1: Smart Contract Development (Backend):** Writing Solidity contracts (CrowdSource.sol, Authentication.sol) to handle logic like adding questions, voting/vetting, and storing hashes.
- **Phase 2: Frontend Interface:** Building the user dashboards (Contributor, Teacher, Exam Center) using HTML/JS/Bootstrap to interact with the blockchain.
- **Phase 3: Integration (Web3):** Connecting the frontend to the blockchain using **Web3.js** and **Metamask** so users can sign transactions (e.g., "Submit Question").
- **Phase 4: Security & Storage:** Implementing the **IPFS** node for file storage and the **AES/RSA** encryption modules for the final paper.

ACTIVITY DIAGRAM FOR THE REPO PROJECT:



RELEVANT SCREENSHOT OF CODE AND OUTPUT OF ACTIVITY DIAGRAM:

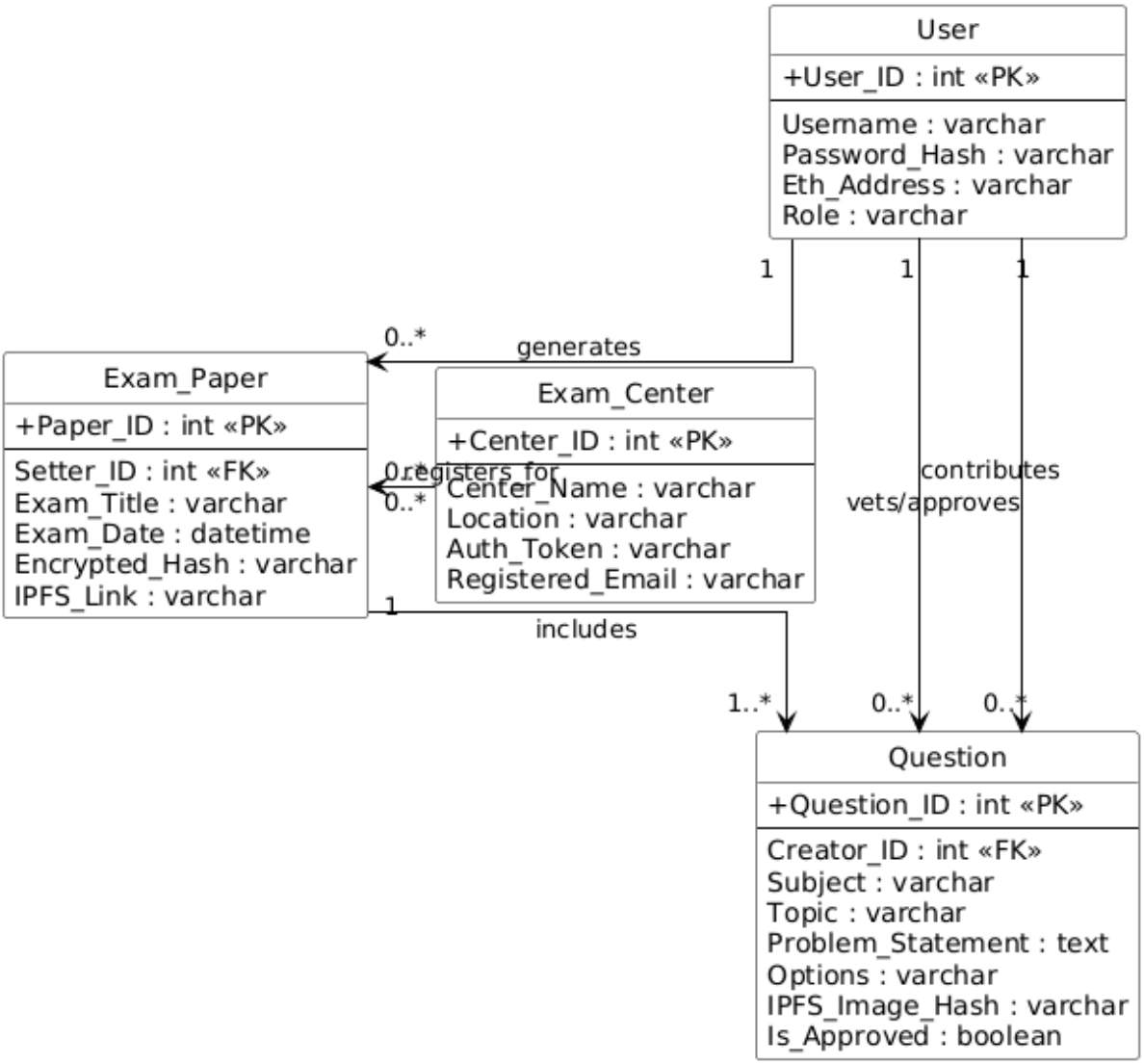


Activity Diagram Description

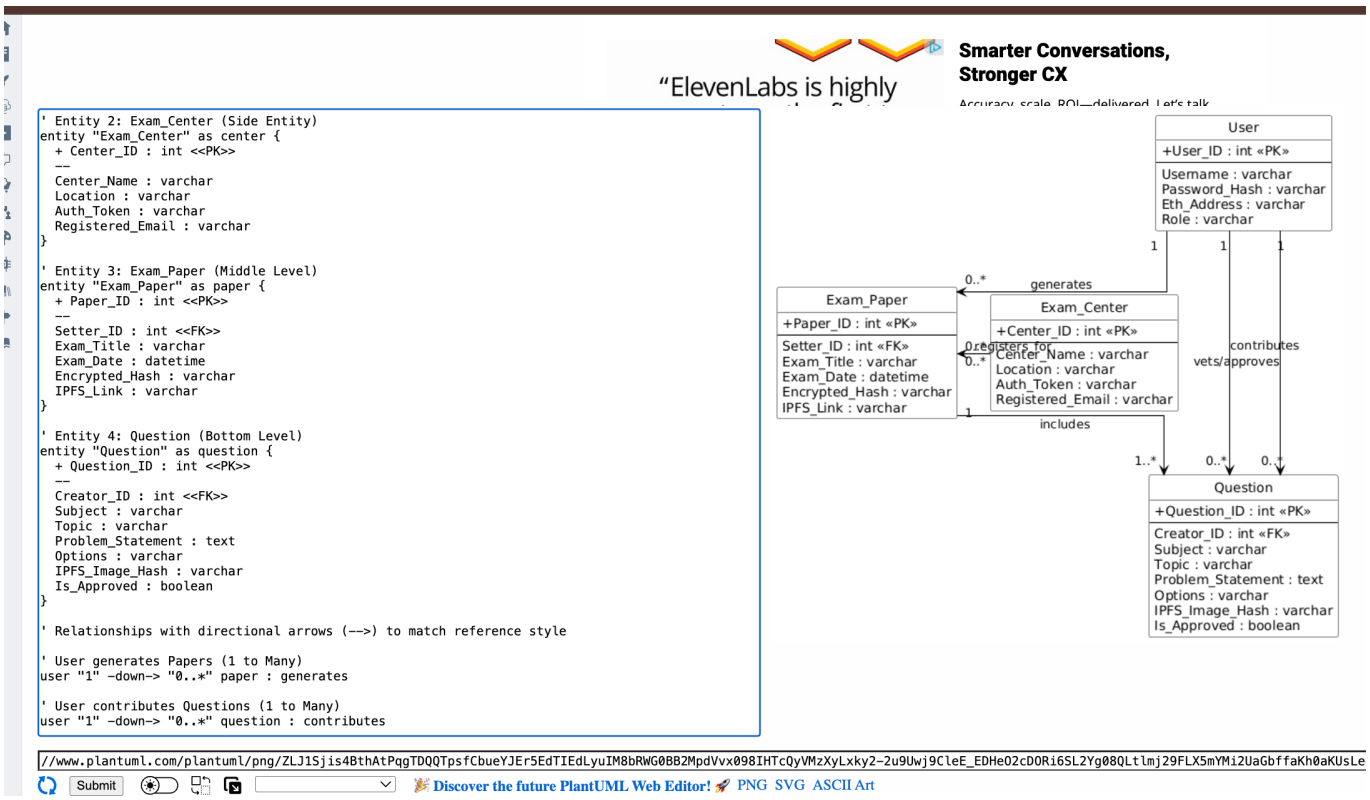
This diagram models the dynamic workflow of the "Secure Exam Delivery" feature, showing the sequence of actions from start to finish.

- **The Flow:**
    1. **Start:** The Expert logs in and selects "Schedule Examination".
    2. **Generation:** The system pulls vetted questions from the pool to generate the paper.
    3. **Encryption:** The paper is encrypted (AES) and uploaded to IPFS.
    4. **Distribution:** The IPFS hash is sent to the Blockchain.
    5. **Time-Check:** The Exam Center's system waits. If Current Time == Exam Time, it triggers the **Auto-Download**.
    6. **End:** The paper is decrypted and displayed to students.

ER DIAGRAM OF THE REPO PROJECT:



RELEVANT SCREENSHOT OF CODE AND OUTPUT OF ER:



Entity-Relationship (ER) Diagram Description

The ER diagram represents the data structure of the DApp (Decentralized Application), focusing on the Smart Contract storage state.

- **Core Entities:**
  - **User:** Stores stakeholder details like Username, Password Hash, and Ethereum Address. It manages roles (Contributor vs. Expert).
  - **Question (Problem):** The central entity containing the Subject, Topic, Question Text, Options, Image Hash (for diagrams), and Approval Status. It links back to the **User** who created it.
  - **Exam_Paper:** Represents the aggregated final paper. It links to the specific **Questions** selected for that exam and holds the Encrypted Hash for secure distribution.

- **Exam_Center:** Represents the authorized destination that "registers" for a paper and receives the decryption key.

- **Key Relationship:** A **User** contributes many **Questions**; a **Question** belongs to one **Exam_Paper**; an **Exam_Center** registers for a specific **Exam_Paper**.

Comparative Analysis: SecuExam vs. CrowdQuest

Feature / Aspect	SecuExam (our Project)	CrowdQuest (Reference Project)
1. Primary Objective	**Secure Delivery (Last-Mile Security):** Focuses strictly on the safe transport of an *existing* exam paper from the setter to the center without leakage.	**Crowdsourced Creation (End-to-End):** Focuses on creating a question bank via crowdsourcing, vetting questions, and *generating* the paper before securing it.
2. System Architecture	**Centralized (Client-Server):** Uses a standard web architecture with a central server (e.g., Python/Django) managing all logic and data.	**Decentralized (Web3 / DApp):** Uses a Blockchain architecture (Polygon/Ethereum) where logic is handled by Smart Contracts and no single server holds the data.
3. Question Paper Origin	**File Upload:** The professor/setter uploads a complete, pre-made PDF or Word document	**System Generated:** Questions are contributed individually by stakeholders, vetted by experts, and then the system *compiles* them into a paper.
4. Storage Mechanism	**Cloud Database:** Encrypted files and logs are stored in a standard cloud database (like PostgreSQL) or file storage (AWS S3).	**IPFS (InterPlanetary File System):** The exam paper is stored on a decentralized file network (IPFS), while only the *hash* (address) is stored on the Blockchain.
5. Security Logic	**Shamir's Secret Sharing + AES-256:** Uses a specific cryptographic scheme to split the decryption key among multiple officials to prevent insider threats	**Public/Private Key (RSA) + AES:** Relies on the user's Blockchain Wallet (like MetaMask) and Smart Contract permissions for access and decryption.

Feature / Aspect	SecuExam (our Project)	CrowdQuest (Reference Project)
6. Traceability & Accountability	**Dynamic Watermarking:** Embeds the receiver's IP address and timestamp directly onto the footer of the downloaded PDF to trace leaks.	**Immutable Ledger:** Relies on the Blockchain transaction history to prove who accessed or downloaded the paper (transparent audit trail).
7. Time-Lock Mechanism	**Server-Side Timestamp:** The central server checks its internal clock against the exam start time before allowing a download (e.g., if time > 9:30 AM).	**Smart Contract Trigger:** The download logic is likely governed by the Blockchain's block timestamp or an oracle, releasing the key only when the condition is met.
8. Cost Model	**Hosting Costs:** Standard monthly fees for cloud servers (e.g., Heroku/AWS)[8].	**Gas Fees:** Every transaction (uploading a question, vetting, deploying contract) requires paying "Gas" (cryptocurrency transaction fees).

1. Executive Summary & Core Philosophy

While both projects aim to secure the examination process, they address different stages of the exam lifecycle and utilize fundamentally different architectural philosophies.

- **SecuExam (Your Project)** is a **Delivery-Centric System**. Its primary focus is the "Last Mile" security—ensuring that a question paper created by a professor reaches the exam center without leakage. It assumes the paper already exists and focuses entirely on the secure pipeline (Encryption -> Storage -> Time-Locked Delivery). It operates on a **Centralized Trust Model** where the university server is the ultimate authority[1].
- **CrowdQuest** is a **Creation-Centric System**. It focuses on the "First Mile"—crowdsourcing questions from various stakeholders to generate a paper automatically. Its scope includes the creation of the question bank itself. It operates on a **Decentralized Trust Model (Web3)**, using Blockchain to remove the need for a central authority to hold the papers, relying instead on distributed storage (IPFS) and Smart Contracts.

2. Architectural & Logical Differences

This section compares the structural logic of how both systems handle data and security.

A. Centralized vs. Decentralized Architecture

- **SecuExam:** Uses a classic **Client-Server Architecture**.
  - **Logic:** A central server (likely Python/Django or Flask) acts as the gatekeeper. It holds the database of users, schedules, and the encrypted files.
  - **Trust:** The system relies on the security of the central server. If the server administrator is compromised, the papers are at risk. To mitigate this, you utilize **Shamir's Secret Sharing** to split the decryption keys, ensuring no single admin has full access.
  - **Storage:** Uses Cloud Storage (AWS/Heroku) and a relational database (PostgreSQL) for structured data like user logs and schedules.
- **CrowdQuest:** Uses a **Blockchain Architecture (DApp)**.
  - **Logic:** Smart Contracts (Solidity) act as the gatekeeper. The logic is immutable and deployed on the Ethereum/Polygon network.
  - **Trust:** The system relies on cryptographic proofs. No single admin "holds" the paper; it is distributed across the network.
  - **Storage:** Uses **IPFS (InterPlanetary File System)** for storing the actual question papers. Only the *hash* (the address) of the paper is stored on the Blockchain, making it tamper-proof but decentralized.

B. The "Time-Lock" Mechanism

- **SecuExam (Server-Side Check):**
  - **Requirement:** The server checks its own system time against the Exam Start Time.
  - **Logic:** IF (Current_Time >= Exam_Start_Time - 30 mins) THEN Allow_Download. This relies on NTP (Network Time Protocol) synchronization
  - **Pros:** Fast, reliable, and easy to implement using Cron jobs
- **CrowdQuest (Smart Contract Trigger):**
  - **Requirement:** The Blockchain block timestamp determines access.

- **Logic:** The Smart Contract has a condition that releases the decryption key or IPFS hash only when a specific block time is reached.
- **Pros:** Immutable; no admin can "change the clock" to leak the paper early.
- **Cons:** Can be susceptible to minor time drifts in block validation or high gas fees to trigger the release.

3. Comparison of Security Logic

A. Encryption & Key Management

- **SecuExam:**
  - **Method: AES-256** for file encryption.
  - **Key Management:** You employ **Shamir's Secret Sharing**. This is a highly sophisticated feature where the master key is split into "fragments." A single person cannot decrypt the paper; a quorum (e.g., 3 out of 5 officials) must combine their fragments to reconstruct the key. This specifically targets "Insider Threats".
  - **Traceability:** Implements **Dynamic Watermarking**. If a leak *does* occur, the file contains the downloader's IP address embedded in the footer, allowing post-leak forensics.
- **CrowdQuest:**
  - **Method: RSA + AES** Hybrid Encryption.
  - **Key Management:** Relies on Public/Private key pairs associated with the user's Blockchain Wallet (Metamask). Access is granted if the user's wallet address is whitelisted in the Smart Contract.
  - **Traceability:** Relies on the immutable ledger. Every interaction (upload/download) is a transaction on the blockchain, creating a permanent, unalterable audit trail.

B. Input Validation & Paper Generation

- **SecuExam:**
  - **Input:** Accepts PDF/Word files up to 50MB directly from the Paper Setter[9]. It assumes the setter has already created the paper offline.

    - **Validation:** Checks file size and format but does not check the *content* of the questions.
- **CrowdQuest:**
    - **Input:** Granular questions entered into a web form (text, images, options).
    - **Validation:** Uses a "Vetting System" where experts approve/reject specific questions before they enter the pool. The paper is then *generated* by the system, not uploaded.

4. Limitations & Risk Analysis

SecuExam Limitations (Our Project)

1. **Single Point of Failure:** Since it relies on a central cloud server (AWS/Heroku), a server crash or DDoS attack during the critical 30-minute window before the exam could prevent *all* centers from downloading the paper
2. **Admin Trust:** While Shamir's Sharing mitigates this, a "Super Admin" with root access to the server might theoretically bypass application-level logic (though this is a standard risk in Web2 apps).
3. **Static Content:** If an error is found in the question paper after upload, the setter must upload a completely new file and re-encrypt it. There is no dynamic editing of individual questions.

CrowdQuest Limitations

1. **Complexity & Cost:** Deploying on Blockchain requires "Gas Fees" for every transaction (uploading questions, vetting, generating papers). This can make the system expensive to run compared to a standard database.
2. **Latency:** Retrieving data from IPFS and waiting for Blockchain transaction confirmations can be slower than a direct SQL query, which is not ideal for last-minute exam pressure.
3. **Data Privacy:** While encrypted, the metadata (who transacted with whom) is visible on the public ledger (unless a private blockchain is used), which might reveal exam schedules or setter identities to sophisticated observers.

Verdict: Our **SecuExam** project is better scoped for a Software Engineering Lab as it focuses deeply on one critical problem (Safe Delivery) and implements specific, high-value security features (Time-Lock, Watermark, Key Splitting) rather than trying to build an entire ecosystem.